

UT 5. Sesiones, Cookies y Autenticación (15 horas)

Manual Didáctico Didactizado • Teoría Completa, Esquemas Gráficos, Código Fuente y Ejercicios Prácticos

PRESENTACIÓN DE LA UNIDAD DIDÁCTICA 5

Bienvenido a la quinta unidad del módulo DWES. Como vimos en la UT1, el protocolo HTTP es 'sin estado' (stateless), lo que significa que el servidor olvida quién es el cliente entre una petición y la siguiente. En esta unidad aprenderemos a mantener la persistencia de estado mediante Cookies y Sesiones. Estudiaremos cómo crear y proteger cookies con banderas de seguridad (Secure, HttpOnly, SameSite), comprenderemos el mecanismo de sesiones de PHP con PHPSESSID y \$_SESSION, prevendremos ataques de secuestro como Session Fixation regrabando identificadores, diseñaremos un Flujo de Autenticación Completo con hashing seguro de contraseñas (bcrypt/Argon2) y protegeremos rutas privadas implementando Control de Acceso Basado en Roles (RBAC) mediante Middlewares caseros.

5.1. Gestión de Cookies

Una Cookie es un pequeño fragmento de información (texto) que el servidor web envía al navegador del cliente para que este lo guarde en el disco duro y lo reenvíe automáticamente en cada petición posterior al mismo dominio.

Concepto de Cookie y Envío de Cabeceras (`Set-Cookie`)

Para crear o modificar una cookie desde PHP utilizamos la función nativa `setcookie()`. Es vital recordar que las cookies se envían mediante cabeceras HTTP (`Set-Cookie:`), por lo que debemos llamar a `setcookie()` ANTES de generar cualquier salida de texto o HTML hacia la pantalla.

Lectura a través de `$_COOKIE`, Expiración, Dominio y Ruta

- Lectura: Los datos enviados por el navegador están disponibles automáticamente en el array superglobal `$_COOKIE['nombre']`.
- Expiración: Se define como una marca de tiempo Unix (Timestamp). Si se omite o vale 0, la cookie caduca al cerrar el navegador (Cookie de Sesión).
- Ruta y Dominio: Especifican en qué URLs o subdominios de la web el navegador tiene permiso para enviar la cookie.

Atributos de Seguridad (`Secure`, `HttpOnly`, `SameSite`)

Guardar información sensible en cookies sin protección es un peligro grave de seguridad. PHP 7.3+ permite definir las cookies utilizando un array de opciones de configuración para activar las banderas de protección:



Figura 5.1: Atributos de seguridad esenciales para la protección de Cookies.

- Secure: Exige que la cookie solo viaje si la conexión es HTTPS cifrada.
- HttpOnly: Oculta la cookie del alcance de JavaScript (document.cookie). Blinda la cookie contra robos por ataques Cross-Site Scripting (XSS).
- SameSite: Previene ataques de Falsificación de Petición en Sitios Cruzados (CSRF). Acepta los valores 'Lax' (recomendado) o 'Strict'.

```
// ejemplo_gestion_cookies.php
<?php
declare(strict_types=1);

// Creación de una cookie segura en PHP 7.3+ / PHP 8+
$nombreCookie = 'preferencia_tema';
$valorCookie = 'oscuro';
$tiempoExpiracion = time() + (86400 * 30); // 30 días de duración

setcookie($nombreCookie, $valorCookie, [
    'expires' => $tiempoExpiracion,
    'path'    => '/',
    'domain'  => 'localhost',
    'secure'  => false, // Poner true en producción con HTTPS
    'httponly' => true, // Inaccesible desde JavaScript (Protección Anti-XSS)
    'samesite' => 'Lax' // Protección Anti-CSRF
]);

// Lectura de la cookie en peticiones posteriores
$temaActual = $_COOKIE['preferencia_tema'] ?? 'claro';
echo "El tema seleccionado por el usuario es: " . htmlspecialchars($temaActual);
?>
```

5.2. Manejo de Sesiones

Debido a que las cookies se guardan en la máquina del usuario y pueden ser manipuladas fácilmente, NUNCA debemos almacenar información sensible en ellas (como rol de usuario, contraseñas o datos bancarios). La solución profesional es utilizar **Sesiones**.

Mecanismo de Sesión e Identificador `PHPSESSID`

Una sesión guarda los datos reales en el disco duro del SERVIDOR. Al cliente solo se le envía un identificador aleatorio único denominado `PHPSESSID` almacenado en una cookie de sesión. En cada petición, el navegador presenta su `PHPSESSID` y PHP recupera los datos correspondientes del servidor.



Figura 5.2: Arquitectura del mecanismo de Sesión entre Navegador y Servidor.

Inicio con `session\_start()` y Almacenamiento en `\$\_SESSION`

Para iniciar o reanudar una sesión existente debemos invocar la función `session\_start()` en la PRIMERA LÍNEA del archivo PHP (antes de renderizar ningún HTML). Una vez iniciada, podemos guardar y leer cualquier variable en el array superglobal `\$\_SESSION`.

```
// ejemplo_inicio_sesion.php
<?php
declare(strict_types=1);

// Debe ser la primera instrucción antes de cualquier salida HTML
session_start();

// Guardar datos en la sesión del servidor
$_SESSION['usuario'] = 'Yolanda';
$_SESSION['rol'] = 'administrador';
$_SESSION['ultimo_acceso'] = date('Y-m-d H:i:s');

echo "Sesión iniciada correctamente para " . $_SESSION['usuario'];
?>
```

5.3. Seguridad en Sesiones

El secuestro de sesión (**Session Hijacking**) ocurre cuando un atacante consigue robar o adivinar el `PHPSESSID` de una víctima, permitiéndole suplantar su identidad en el sitio web sin conocer su contraseña.

Regeneración de ID de Sesión (`session\_regenerate\_id`)

En el ataque **Session Fixation**, el atacante obliga a la víctima a usar un `PHPSESSID` conocido. Para evitarlo, la norma de seguridad estricta es invocar `session\_regenerate\_id(true)` INMEDIATAMENTE después de que el usuario se autentique o cambie de nivel de privilegio. Esto genera un nuevo identificador y destruye el anterior de la memoria.



Figura 5.3: Prevención del ataque Session Fixation mediante `session_regenerate_id()`.

Destrucción de Sesión y Cierre de Sesión Seguro (Logout)

Para cerrar la sesión de un usuario de forma 100% segura no basta con borrar el array. Se deben ejecutar 4 pasos clave:

1. Vaciar el array en memoria: `$_SESSION = [];`
2. Destruir la cookie del navegador caducándola en el pasado: `setcookie(session_name(), '', time() - 3600, ...);`
3. Destruir el fichero físico en el servidor: `session_destroy();`
4. Redirigir al usuario al login.

```
// ejemplo_logout_seguro.php
<?php
declare(strict_types=1);
session_start();

// 1. Vaciar todas las variables de sesión
$_SESSION = [];

// 2. Destruir la cookie de sesión del navegador
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
```

```

        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// 3. Destruir la sesión en el servidor
session_destroy();

// 4. Redirigir a la página de login
header('Location: login.php');
exit();
?>

```

5.4. Sistema de Autenticación Completo

El proceso de autenticación valida la identidad de un usuario. Regla sagrada de seguridad: NUNCA guardes contraseñas en texto plano ni utilices algoritmos obsoletos (MD5 o SHA1).

Hashing Seguro de Contraseñas con `bcrypt` / `Argon2`

Un **Hash** es un algoritmo matemático unidireccional (irreversible). PHP incluye funciones nativas que aplican salt dinámico y algoritmos robustos como `bcrypt` o `Argon2id`:

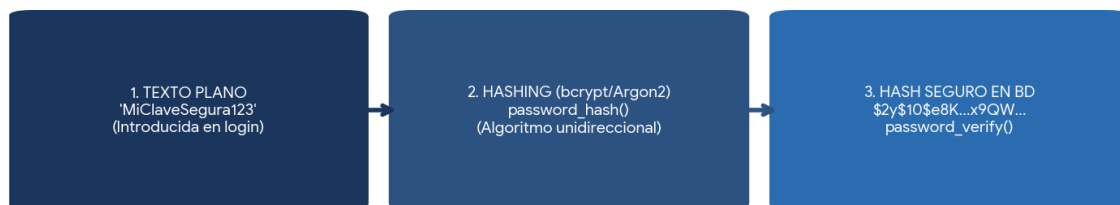


Figura 5.4: Flujo de cifrado seguro con `password_hash()` y verificación con `password_verify()`.

- `password_hash($pass, PASSWORD_DEFAULT)`: Genera un hash seguro y robusto. Aplica automáticamente un 'salt' aleatorio para evitar ataques por tablas de arcoíris.
- `password_verify($passIngresada, $hashBD)`: Compara la clave en texto plano con el hash guardado en la base de datos. Devuelve true si coinciden.

```

// ejemplo_registro_login_seguro.php
<?php
declare(strict_types=1);
require_once 'conexion.php'; // Obtenemos $pdo

// REGISTRO DE USUARIO

```

```

function registrarUsuario(string $email, string $password, PDO $pdo): bool {
    // Generamos el hash seguro de la contraseña usando el algoritmo por defecto
    (bcrypt)
    $hashPassword = password_hash($password, PASSWORD_DEFAULT);

    $sql = "INSERT INTO usuarios (email, password_hash, rol) VALUES (:email, :pass,
'alumno')";
    $stmt = $pdo->prepare($sql);
    return $stmt->execute([':email' => $email, ':pass' => $hashPassword]);
}

// LOGIN DE USUARIO
function iniciarSesion(string $email, string $password, PDO $pdo): bool {
    $sql = "SELECT id, email, password_hash, rol FROM usuarios WHERE email = :email";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([':email' => $email]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    // Verificamos si existe el usuario y si la contraseña coincide con el hash
    if ($user && password_verify($password, $user['password_hash'])) {
        session_start();
        session_regenerate_id(true); // Previene Session Fixation
        $_SESSION['usuario_id'] = $user['id'];
        $_SESSION['email'] = $user['email'];
        $_SESSION['rol'] = $user['rol'];
        return true;
    }
    return false;
}
?>

```

5.5. Control de Accesos

Una vez autenticado el usuario, debemos controlar a qué secciones o páginas de la aplicación tiene permiso para acceder. El modelo estándar es el **Control de Acceso Basado en Roles (RBAC)**.

Control de Acceso Basado en Roles (RBAC) y Middleware Casero

Un **Middleware** es una capa intermedia que se ejecuta ANTES de cargar el contenido de una página privada. Comprueba si el usuario tiene una sesión activa y si posee el rol requerido (ej. 'admin'). Si no cumple las condiciones, intercepta la petición y redirige al usuario a la pantalla de login o muestra un error 403 Forbidden.

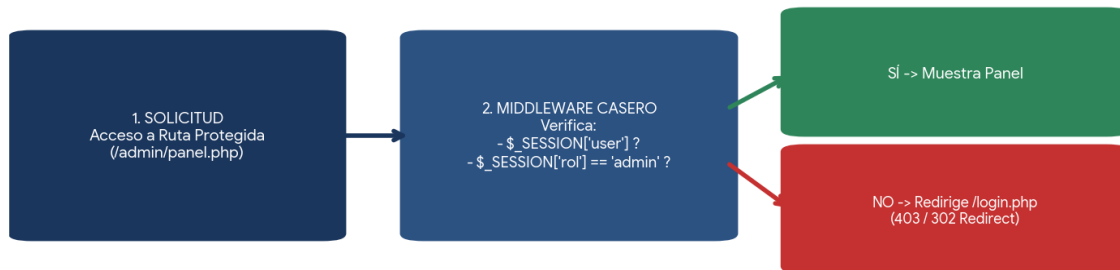


Figura 5.5: Intercepción y validación de permisos mediante un Middleware de control de acceso.

```
// ejemplo_middleware_rbac.php
<?php
declare(strict_types=1);

// Middleware casero de protección de rutas privadas
function verificarAcceso(array $rolesPermitidos = []): void {
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }

    // 1. Verificamos si el usuario ha iniciado sesión
    if (!isset($_SESSION['usuario_id'])) {
        header('Location: login.php?error=no_autenticado');
        exit();
    }

    // 2. Verificamos si el rol del usuario está autorizado
    if (!empty($rolesPermitidos) && !in_array($_SESSION['rol'], $rolesPermitidos,
true)) {
        http_response_code(403); // HTTP 403 Forbidden
        echo "<h2>[403 Acceso Denegado]: No tienes permisos de '" . implode(', ',
$rolesPermitidos) . "' para ver esta sección.</h2>";
        exit();
    }
}

// USO EN UNA PÁGINA PRIVADA DE ADMINISTRACIÓN (admin/panel.php):
// require_once 'middleware.php';
// verificarAcceso(['admin', 'profesor']); // Solo permite entrar a admin o profesor
?>
```

6. Batería de Ejercicios Finales de la Unidad 5

A continuación se presenta la batería de actividades prácticas de la UT5 graduadas para afianzar las competencias de persistencia de estado con cookies seguras, manejo de sesiones, autenticación con password_hash y protección RBAC.

Ejercicio 1 (Guiado): Gestor de Preferencias de Usuario con Cookies Seguras (2 Horas)

- **Instrucciones paso a paso:** 1. Crea un formulario en `preferencias.php` con un selector para elegir el color de fondo de la web ('claro', 'oscuro', 'azul') y el idioma.
- 2. Procesa el formulario guardando las selecciones en cookies seguras utilizando la sintaxis de array de opciones de `setcookie()` con `httponly => true` y `samesite => 'Lax'`.
- 3. Haz que la página aplique dinámicamente los estilos CSS recibidos de `\$\_COOKIE` en cada recarga.
- 4. Añade un botón 'Restablecer' que elimine las cookies caducándolas en el pasado (`time() - 3600`).

Ejercicio 2: Carrito de Compras Persistente en Sesión (3 Horas)

- **Requisitos del programa:** Crea un script `carrito.php` que inicie sesión con `session\_start()`.
- Si no existe la clave `\$\_SESSION['carrito']`, inicialízala como un array vacío.
- Implementa tres acciones simuladas enviadas por GET/POST: `agregar`, `eliminar\_item` y `vaciar\_carrito`.
- Renderiza una tabla HTML formateada mostrando los productos en la cesta, la cantidad y el precio total calculado dinámicamente.

Ejercicio 3: Módulo Completo de Registro y Login con Password Hash (4 Horas)

- **Requisitos del programa:** Crea la base de datos `autenticacion\_db` con la tabla `usuarios` ('id', 'nombre', 'email' UNIQUE, 'password_hash', 'rol').
- Crea la página `registro.php` que sanitice las entradas y guarde la clave procesada con `password\_hash(\$pass, PASSWORD\_DEFAULT)`.
- Crea la página `login.php` que valide el correo y compruebe el hash con `password\_verify()`.
- Al autenticar con éxito, invoca `session\_regenerate\_id(true)` y guarda los datos en `\$\_SESSION`.

Ejercicio 4 (Práctica Evaluatoria Global): Sistema de Gestión Académica Privado con Roles y Middleware RBAC (6 Horas)

- **Consigna del Proyecto:** Objetivo: Desarrollar un portal académico protegido por contraseña y con restricciones de acceso por rol.
- 1. Desarrolla el archivo `middleware.php` con la función `verificarAcceso(array \$rolesPermitidos)`.
- 2. Crea tres páginas privadas protegidas por el middleware:
 - `panel\_alumno.php` (Acceso para roles: 'alumno', 'profesor', 'admin').
 - `panel\_profesor.php` (Acceso para roles: 'profesor', 'admin').
 - `panel\_admin.php` (Acceso EXCLUSIVO para rol: 'admin').

- 3. Si un alumno intenta acceder a `panel\_admin.php`, el middleware debe bloquear la petición, emitir un código HTTP 403 Forbidden y mostrar una caja de alerta roja.
- 4. Implementa la página `logout.php` realizando el cierre de sesión seguro en 4 pasos (vaciar `\$\_SESSION`, caducar cookie `PHPSESSID`, `session\_destroy()` y redirección).

Criterios de Evaluación Oficiales

RÚBRICA DE EVALUACIÓN MÓDULO DWES - UT5

CE 5.a: Se han creado y leído cookies configurando sus atributos de expiración, ruta, dominio y banderas de seguridad (Secure, HttpOnly, SameSite).

CE 5.b: Se ha gestionado el almacenamiento persistente de datos en el servidor utilizando el array superglobal `\$\_SESSION` y `session\_start()`.

CE 5.c: Se han aplicado medidas de seguridad en sesiones regenerando identificadores con `session\_regenerate\_id()` y cerrando sesiones de forma limpia.

CE 5.d: Se ha implementado un flujo completo de autenticación hashing contraseñas con `password\_hash()` y verificándolas con `password\_verify()`.

CE 5.e: Se ha restringido el acceso a rutas privadas mediante la implementación de Control de Acceso Basado en Roles (RBAC) y middlewares.